

Executable Mathematics

From Numerical Exercises to Modular Computational Tools

T. R. Bentley

Repository: Tybent18/math-for-computing

Document class: repository-grounded technical report

Edition: September 2026

Scope

Computational mathematics, numerical exercises, and modular domain utilities.

Evidence statement

Claims in this report are limited to repository artifacts identified by filename. Proposed tests, benchmarks, integrations, and research directions are labeled as future work; they are not reported results.

Abstract

This report traces a mathematical-programming collection from standalone numerical exercises to a modular Python toolkit spanning calculus, linear algebra, number theory, combinatorics, sequences, chaos, physics, and finance. It emphasizes the translation of definitions into executable procedures, the importance of validation and numerical assumptions, and the distinction between implemented functions and experimentally verified accuracy.

1. Repository evidence map

Cluster	Artifacts	Interpretation
Modular toolkit	advanced_math_utilities/	Domain modules for calculus, linear algebra, number theory, combinatorics, sequences, chaos, physics, and finance.
Combinatorics	combination.c, permutation.c	Discrete counting procedures.
Sequences and approximation	collatz_sequence.c, fibonnaci.py, exponential.c, partial_sum.c	Iterative sequences and numerical approximation.
Statistics and algebra	standard_deviation.c, quad_root.c	Focused statistical and equation-solving exercises.
Representation	binary_convert.java, array_pairwise_swap.c	Numeric representation and array transformation.

2. Technical framing

- How faithfully does code encode the mathematical definition?
- Where do precision, stability, and domain constraints matter?
- How does modularization change reuse and validation?

3. Reproducible inspection protocol

A reviewer should clone the repository, record the commit SHA and toolchain versions, inspect each mapped artifact, execute only examples whose entry points and prerequisites are declared, preserve standard output and errors, and compare observations with documented expectations. Failures in deliberate debugging targets are evidence only when the expected failure class is stated in advance.

4. Evidence maturity

Level	Meaning	Current interpretation
E0	Artifact listed	Source file exists in the repository
E1	Behavior described	README or report states intended behavior
E2	Execution recorded	Command, environment, and output are retained

E3	Behavior tested	Repeatable assertions evaluate expected behavior
E4	Comparative result	Frozen data supports a bounded comparison

5. Limitations

- No unified accuracy benchmark is currently recorded.
- Standalone programs vary in interface and validation behavior.
- Scientific reliability requires reference values, tolerances, and environment capture.

6. Development roadmap

- Add reference-value and property-based tests.
- Document numeric domains and error tolerances.
- Benchmark accuracy separately from execution speed.

7. Portfolio role

This repository belongs to a cumulative foundation sequence: Foundations & Algorithms → Data Structures Practice → OOP Concepts → Math for Computing → Practical Utilities. The sequence demonstrates increasing abstraction and application; advanced repositories carry the stronger systems and empirical-research claims.

Conclusion

The repository is most credible when its modest scale is treated as a strength: each artifact can be inspected directly, every claim can point to code, and future measurements can be added without rewriting history. Sophistication comes from traceability, not inflated labels.